
TEMA 2: INTRODUCCIÓN C++

INICIALIZAR

```
#include <iostream>

using namespace std;

int main () {

    ...

    return 0;

}
```

INTRODUCCIÓN Y EXPRESIÓN DE DATOS

```
cin >> x;                                cin.get(c); (Obteniendo caracteres espacios)

cout << x;
```

MODIFICACIÓN Y COMPARACIÓN DE DATOS

x++; (x=x+1)	x *= a; (x = x*a)
x--; (x=x-1)	x /= a; (x = x/a)
x += a; (x = x+a)	x %= a; (x = x%a)
x -= a; (x = x-a)	==, >, =, <=, !=

ESTRUCTURAS DE SELECCIÓN

IF

```
if (condición) {  
  
    //Acción  
  
}
```

ELSE (IF)

```
if (condición_1) {  
  
    //Acción_1  
  
} else if (condición_2) {  
  
    //Acción_2}  
  
} else {  
  
    //Acción_2}
```

ESTRUCTURAS DE ITERACIÓN

FOR

```
for (int i = 0; i < límite; i++) {  
  
    //Acción  
  
}  
  
for (int i = límite; i > 0; i--) {  
  
    //Acción  
  
}
```

DO WHILE

```
do {  
  
    //Acción  
  
} (condición);
```

WHILE

```
while (condición) {  
  
    //Acción  
  
}
```

TEMA 3: FUNCIONES Y PROCEDIMIENTOS

LEER (PROCEDIMIENTOS)

```
void leer (int& a) {  
  
    cout << "Di un numero ";  
  
    cin >> x;  
  
}
```

SUMA (FUNCIONES)

```
int suma (int a, int b) {  
  
    int suma;  
  
    suma = a + b;  
  
    return suma;  
  
}
```

PRIMO (FUNCIONES)

```
bool esPrimo (int num) {  
  
    bool res = false;  
  
    int divisor, tope;  
  
    if (num >= 2) {  
  
        tope = num / 2;  
  
        divisor = 2;  
  
        while ((divisor <= tope) && (num % divisor != 0)) {  
  
            divisor++;  
  
        }  
  
        res = divisor > tope;  
  
    }  
  
    return res;  
  
}
```

TEMA 4: TIPOS ESTRUCTURADOS

REGISTROS

```
#include <iostream>

using namespace std;

struct R(registro)_X {

    tipos de variables;

    // Para acceder: nombreinicializado.tipos de variables

}

int main () {

    R(registro)_X nombreinicializado;

    cin >> nombre.variable1;

    cin >> nombre.variable2;

    return 0;

}
```

REGISTROS DE ARRAYS

```
#include <iostream>

#include <array>

using namespace std;

const int MAX = 20;

typedef array<tipo, MAX> Nombre_Array;

struct R(registro)_X {

    Nombre_Array Array;

    // Para acceder: nombreinicializado.array

}
```

INICIALIZAR ARRAYS

```
#include <iostream>
```

```
#include <array>
```

```
using namespace std;
```

```
const int MAX = 20;
```

ARRAYS COMPLETOS

```
typedef array<tipo, MAX> Nombre_Array;
```

ARRAYS INCOMPLETOS

```
typedef array<tipo, MAX> Nombre_Array;
```

```
struct Nombre_Registro {
```

```
    Nombre_Array Nombre_Array_Registro;
```

```
    int tam;
```

```
};
```

```
// Cuando se inicializa, marcar tam como el límite deseado y tenerlo en cuenta en los bucles.
```

MATRICES (ARRAYS BIDIMENSIONALES)

```
const int NFILAS = 5;
```

```
const int NCOLUMNAS = 5;
```

```
typedef array<tipo, NCOLUMNAS> TFila;
```

```
typedef array<TFila, NFILAS> TMatriz;
```

LECTURA DE ARRAYS

```
void leerarray_completo (Nombre_Array& A) {  
  
    for (int f = 0; f < MAX; f++) {  
  
        cin >> A[f];  
  
    }  
  
}
```

MATRICES

```
void leermatriz_incompleta (RMatriz& m) {  
  
    do {  
  
        cout << "Introduzca dimension de la matriz cuadrada (maximo " << MAX << "): ";  
  
        cin >> m.tam;  
  
    } while ((m.tam < 1) || (m.tam > MAX));  
  
    cout << "Introduzca la matriz fila a fila:\n";  
  
    for (int f = 0; f < m.tam; f++) {  
  
        for (int c = 0; c < m.tam; c++) {  
  
            cin >> m.datos[f][c];  
  
        }  
  
    }
```

INCREMENTO CIRCULAR (PROCEDIMIENTO)

```
void incrementoCircular (int& ind, int max) {  
  
    ind = (ind + 1) % max;  
  
}
```

STRING

INICIALIZAR

```
#include <iostream>

#include <array>

#include <string>

using namespace std;
```

```
int main () {

    string pal;

    cin >> pal;

    return 0;

}
```

RECORRER STRING (PROCEDIMIENTO)

```
void (string& pal) {

    for (int i = 0; i < int(pal.size()); i++) {

    }}

}}
```

CONCATENAR (PROCEDIMIENTO)

```
void (string& pal) {

    cin >> newpal;

    pal = pal + newpal;

}}
```